LECTURE 8

Transformers: An Intuitive Introduction

Reference Text

Language models, meaning as numbers, contextual embeddings and attention — built from analogies and checked by hand

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 How to Use This Text **3**

2 What a Language Model Does **3**
2.1 One job: predict the next token . . . 3
2.2 Writing a whole answer . . . 3

3 Meaning as Numbers **4**
3.1 Why an ID number is not enough . . . 4
3.2 Embeddings: coordinates on a meaning map . . . 4
3.3 Measuring closeness: cosine similarity . . . 4
3.4 Directions carry meaning . . . 4
3.5 Where embeddings come from . . . 5

4 Static versus Contextual Embeddings **5**
4.1 The battery problem . . . 5
4.2 Context can come from far away . . . 5

5 The Encoder and the Decoder **5**
5.1 Two halves . . . 5
5.2 Training and inference . . . 6
5.3 BERT, GPT and the original design . . . 6

6 Inside the Encoder: Tokens and Positions **6**
6.1 Step 1 — tokens and IDs . . . 6
6.2 Step 2 — embedding lookup . . . 7
6.3 Step 3 — position . . . 7

7 Attention **7**
7.1 Which words should update which? . . . 7
7.2 Query, key and value: the radio-net analogy . . . 7
7.3 Where queries, keys and values come from . . . 8
7.4 The recipe in four steps . . . 8
7.5 Worked example: the battery arithmetic . . . 8

8 Multi-Head Attention **9**

9 Completing the Block **10**
9.1 Residual connections . . . 10
9.2 Layer normalisation . . . 10
9.3 The feed-forward network . . . 10
9.4 One block, then stack it . . . 10

10 The Decoder **11**
10.1 Writing one token at a time 11
10.2 Masked self-attention 11
10.3 Cross-attention 11
10.4 The whole journey 11

11 Defence Relevance and Cautions **12**

12 Common Misconceptions **12**

13 Summary of Equations **13**

14 Check Your Understanding **13**

1 How to Use This Text

This text accompanies the Lecture 8 slides. It assumes Lecture 6 (a neuron is a weighted sum followed by a non-linearity; backpropagation learns every weight) and uses Lecture 7's idea that a network can learn its own features. The aim is *intuition*: every idea is introduced through an analogy and then shown as the arithmetic that is actually computed. Lecture 9 gives the full mathematics of the Vaswani et al. (2017) architecture. One running example threads through the text: the word **battery**, which means a unit of guns in "the artillery battery fired" and a power cell in "the radio battery died". By Section 7.5 you will have computed, with a calculator, how a transformer tells the two apart.

2 What a Language Model Does

2.1 One job: predict the next token

A **language model** reads the text so far and outputs a probability for every token in its vocabulary — the model's estimate of what comes next. Internally it first produces a *score* (a logit) for each token; the **softmax** function turns scores into probabilities that are positive and sum to 1:

$$P(\text{token } i) = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

Worked example — "Request immediate air ____"

Suppose the scores for five candidates are support 3.1, strike 2.0, cover 1.5, drop −0.5, parade −1.2. The exponentials are 22.198, 7.389, 4.482, 0.607, 0.301, summing to 34.976. Dividing gives

$$P = (0.635,\ 0.211,\ 0.128,\ 0.017,\ 0.009).$$

"Support" is most likely; "parade" is possible but improbable. The probabilities reflect the many similar sentences the model saw in training.

2.2 Writing a whole answer

To write more than one word the model repeats its single job: the chosen token is appended to the text and the longer text is fed back in. "Request immediate air" → *support* → "Request immediate air support" → *at* → ... This loop is called **autoregressive generation**. A chat assistant's long reply is this loop run hundreds of times. The next token may be the most probable one (greedy decoding) or be *sampled* in proportion to the probabilities, which gives more varied text. "Large" language models do the same job with billions of learned parameters trained on very large amounts of text.

Key point

A language model never "looks up" an answer. It repeatedly predicts a probable next token given everything before it. Fluency is what it is trained for; truth is not guaranteed.

3 Meaning as Numbers

3.1 Why an ID number is not enough

A network computes with numbers, so words must become numbers. The simplest scheme numbers the dictionary: *howitzer* 4,118, *hospital* 4,119, *howl* 4,120. The numbers follow spelling, not meaning: nothing in 4,118 says "weapon", and the arithmetic $4{,}119 - 4{,}118 = 1$ suggests that a hospital is "one more" than a howitzer, which is meaningless. A one-hot vector (all zeros with a single 1 in the word's position) removes the false ordering but makes every pair of words equally different.

3.2 Embeddings: coordinates on a meaning map

Definition — embedding

An **embedding** represents a token by a vector of d real numbers, learned so that tokens used in similar contexts receive nearby vectors.

The analogy is a military grid reference: a location is fixed by coordinates, and nearby references are nearby places. On a meaning map *howitzer*, *mortar* and *artillery* cluster together; *hospital*, *clinic* and *medic* form a different cluster; *howl* lies elsewhere, whatever its spelling.

3.3 Measuring closeness: cosine similarity

The usual measure of how alike two embeddings are is the cosine of the angle between them:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|},$$

which is 1 for vectors pointing the same way, 0 for perpendicular vectors and -1 for opposite ones. It ignores length and compares direction only.

Worked example — cosine similarity on the toy map

Use the two-dimensional toy vectors of the running example: artillery $(3, 0.2)$, fired $(2.4, 0.4)$, radio $(0.2, 3)$, died $(0.4, 2.2)$, battery $(1, 1)$.

artillery–fired: $\dfrac{3(2.4) + 0.2(0.4)}{\sqrt{9.04}\sqrt{5.92}} = \dfrac{7.28}{3.007 \times 2.433} = 0.995$ (almost the same direction).

artillery–radio: $\dfrac{0.6 + 0.6}{3.007 \times 3.007} = \dfrac{1.2}{9.04} = 0.133$ (nearly unrelated).

battery–artillery and battery–radio: both 0.753. The static vector for battery sits exactly half-way between the two meanings.

3.4 Directions carry meaning

In a well-trained space, consistent *differences* between words point in consistent directions. On the slide's toy map the step from *advance* $(1.0, 0.2)$ to *advanced* $(1.1, 1.4)$ points the same way as *retreat*→*retreated* and *attack*→*attacked*: a "past-tense" direction. Vector arithmetic exploits this:

$$\overrightarrow{\text{retreat}} - \overrightarrow{\text{advance}} + \overrightarrow{\text{advanced}} = (-1.0, 0.25) - (1.0, 0.2) + (1.1, 1.4) = (-0.9,\ 1.45),$$

and the nearest word is *retreated* at $(-0.95, 1.5)$. Real embeddings show similar directions for singular→plural, country→capital and male→female; Lecture 9 works the last in full.

3.5 Where embeddings come from

Nobody writes embeddings by hand. They start as random numbers; the model reads large amounts of text, tries to predict missing or next words, and backpropagation adjusts every word's vector to improve the predictions. Words that appear in similar company end up close together — the *distributional hypothesis*: "you shall know a word by the company it keeps" (J. R. Firth, 1957). The dimensions have no names: our axes are labelled for teaching only, and in a real model meaning is spread across hundreds or thousands of unlabelled numbers.

Model	numbers per token (d)
word2vec (2013)	300
BERT-base (2018)	768
GPT-2 small (2019)	768
GPT-3 (2020)	12,288

4 Static versus Contextual Embeddings

4.1 The battery problem

word2vec and GloVe produce **static** embeddings: one fixed vector per word, whatever the sentence. Yet "battery" means a unit of guns, a power cell or a criminal offence depending on context, and the words likely to follow differ completely (*fired, displaced, commander* after "artillery battery"; *died, charged, pack* after "radio battery"). A single static point blurs these meanings — as the cosine example showed, it is equally close to both.

Definition — contextual embedding

A **contextual embedding** is a vector for a token *in a particular sentence*: the static vector after it has been updated by information gathered from the surrounding tokens. Producing contextual embeddings is the central job of every transformer layer.

4.2 Context can come from far away

In "The pilot who trained the new cadets for three years finally retired", the one who retired is the pilot, nine words earlier, not the nearer "cadets" or "years". In a report, the clue that "battery" means guns may be a paragraph back ("… trained for months on the new 155 mm guns …"). Older **recurrent neural networks** (RNNs, LSTMs) read text one word at a time and carried a single summary vector forward, so distant clues faded and training could not be parallelised across the sentence. Attention removes that bottleneck: every token can look directly at every other token, however far away, and distance alone does not decide relevance.

5 The Encoder and the Decoder

5.1 Two halves

The original transformer has two parts. The **encoder** reads the whole input at once and produces a contextual vector for every input token: it *understands*. The **decoder** *writes* the output one token at a time, using the encoder's vectors and everything it has written so far. One pattern serves many tasks: free text → structured report, translation, summarisation, question answering.

Worked example — free text to a structured report

Input: "Two enemy tanks moving north near the bridge at 0600." Output: SIZE: 2 / ACTIVITY: moving north / LOCATION: bridge / TIME: 0600 — the opening fields of a SALUTE-style report, written token by token for a human to check.

5.2 Training and inference

Training needs no human labels. From one sentence, "Request immediate air support for the platoon", the model receives six examples: ("Request" → immediate), ("Request immediate" → air), . . . , (". . . for the" → platoon). The next word in real text *is* the label — **self-supervised learning** — so any manual, report or book becomes training data. When the untrained model guesses "drop" instead of "support", the error is measured by cross-entropy, $L = -\ln P(\text{correct token})$, and backpropagated. Across billions of examples this sets the embedding table, the attention matrices and every other weight. At **inference** the trained model is applied to new text with its weights frozen; everything below describes inference. A cadet's years of drills followed by decisions in the field is a fair analogy.

5.3 BERT, GPT and the original design

	Original / T5	BERT-base	GPT-3
Parts used	encoder + decoder	encoder only	decoder only
Numbers per token	512 (original)	768	12,288
Blocks	6 + 6	12	96
Attention heads	8	12	96
Vocabulary	≈37,000	30,522	≈50,000
Training task	translation	fill in masked words	next token
Best at	sequence to sequence	understanding	generating

"Transformer" names the general design; BERT and GPT are particular models built from it. BERT is trained by hiding about 15% of tokens and predicting them from *both* sides; GPT only ever sees the tokens to its left. A GPT-style decoder still builds contextual embeddings — just from leftward context.

6 Inside the Encoder: Tokens and Positions

6.1 Step 1 — tokens and IDs

Text is split into **tokens**. Common words are one token; long or rare words are split into pieces. BERT's tokenizer turns "Reconnaissance team located the battery" into [CLS] recon ##naissance team located the battery [SEP], where "##" marks a continuation. [CLS] (ID 101) opens every input and acts as a summary slot used for classification; [SEP] (ID 102) marks the end of a sentence or the boundary between two. Each token's ID is its row number in the vocabulary. Splitting words into sub-word pieces lets a fixed vocabulary of about 30,000 entries spell any word, including names and jargon never seen in training.

6.2 Step 2 — embedding lookup

The embedding table has one row per vocabulary entry and d columns (30,522 × 768 in BERT-base). Looking up token 6046 simply copies row 6046: no calculation, just an index. At this point "battery" has the same vector in every sentence.

6.3 Step 3 — position

Word order changes meaning: "Alpha relieves Bravo" and "Bravo relieves Alpha" contain the same words with opposite consequences. A transformer processes all tokens *in parallel*, so without extra information it would see a bag of words. The fix is to add a position vector:

$$\text{input}_t = \text{embedding}_t + \text{position}_t.$$

The original paper built position vectors from sine and cosine waves of many frequencies — like reading a clock's second, minute and hour hands together, each position gets a unique, bounded code. Many later models (BERT, GPT-2) learn the position vectors instead. Lecture 9 gives the formula and a worked example.

7 Attention

7.1 Which words should update which?

For each token, attention assigns a **weight** to every token in the sentence (itself included); the weights are positive and sum to 1, and the token's update is the weighted blend of what the others offer. In "recon drone spotted enemy armour near ford", an illustrative set of weights for *armour* is recon 0.04, drone 0.08, spotted 0.22, enemy 0.38, armour 0.14, near 0.05, ford 0.09 (sum 1.00). "Enemy" tells us whose armour; "spotted" what happened to it. Replacing "enemy" by "friendly" would change the contextual vector of "armour" a great deal; replacing "near" by "by" would hardly move it. With n tokens there is an $n \times n$ table of weights.

7.2 Query, key and value: the radio-net analogy

Intuition

a radio net Every station on a net transmits a **call-sign** (its *key*) and has a **message** (its *value*). A listening station tunes for the call-sign it needs (its *query*). The better a transmitted call-sign matches what the listener is tuned for, the louder that station's message comes through. The listener hears every message at its own volume and adds them together; the volume settings always total 100%.

For "battery", the query asks "who tells me my role?"; the key of "artillery" advertises "I name a gun unit"; the key of "the" advertises "I am a determiner". The good match turns "artillery" up and "the" down. In reality the call-signs are not words but learned vectors, and the model learns what to listen for.

7.3 Where queries, keys and values come from

Each token's input vector x (embedding plus position) is multiplied by three learned matrices:

$$q = xW^Q, \qquad k = xW^K, \qquad v = xW^V.$$

W^Q learns how to turn a token into a question, W^K into an advertisement, and W^V into the content it hands over. All three are set in training and fixed at inference. Per attention head they shrink the vector: BERT-base maps 768 numbers to 64, GPT-3 maps 12,288 to 128. Stacking every token's q, k and v as rows gives matrices Q, K and V with one row per token.

7.4 The recipe in four steps

Algorithm — scaled dot-product attention

1. **Match:** score each pair by the dot product $q_i \cdot k_j$.
2. **Scale:** divide by $\sqrt{d_k}$, where d_k is the length of the key vectors.
3. **Softmax:** turn each token's row of scores into weights that sum to 1.
4. **Blend:** output $z_i = \sum_j w_{ij}\, v_j$.

For all tokens at once: $\text{Attention}(Q, K, V) = \text{softmax}\left(\dfrac{QK^\mathsf{T}}{\sqrt{d_k}}\right) V.$

Why K^T? For one query and one key, $q \cdot k$ is one number. With all keys as the rows of K, the matrix product QK^T computes every query–key match at once: entry (i, j) is $q_i \cdot k_j$, an $n \times n$ table.

Why divide by $\sqrt{d_k}$? Dot products grow with vector length: for random vectors with unit-variance components, $q \cdot k$ has variance d_k. Large scores make softmax put almost all weight on one token, its gradients become tiny and learning stalls. For instance the scores $(3.2, 2.0, 2.8)$ give weights $(0.507, 0.153, 0.340)$, but eight times those scores give $(0.961, 0.000, 0.039)$. Dividing by $\sqrt{d_k}$ keeps the scores in a workable range: $\sqrt{64} = 8$ for BERT-base, $\sqrt{128} = 11.3$ for GPT-3.

7.5 Worked example: the battery arithmetic

To compute by hand, take the simplest case $W^Q = W^K = W^V = I$ (the identity), so $q = k = v = x$, and $d_k = 2$.

Worked example — sentence A: "artillery battery fired"

The query is battery's vector $q = (1, 1)$.

key	$q \cdot k$	$\div\sqrt{2}$	$e^{(\cdot)}$	weight
artillery $(3, 0.2)$	3.2	2.263	9.609	**0.458**
battery $(1, 1)$	2.0	1.414	4.113	0.196
fired $(2.4, 0.4)$	2.8	1.980	7.242	0.345
			sum 20.965	1.000

Blend: $z = 0.458(3, 0.2) + 0.196(1, 1) + 0.345(2.4, 0.4) = (2.40,\ 0.43)$.
Residual (add the original): $(1, 1) + (2.40, 0.43) = (3.40,\ 1.43)$.

Worked example — sentence B: "radio battery died"

Scores: radio $(0.2, 3) \cdot (1, 1) = 3.2$, battery 2.0, died $(0.4, 2.2) \cdot (1, 1) = 2.6$. Scaled: 2.263, 1.414, 1.838; exponentials 9.609, 4.113, 6.287, sum 20.009; weights 0.480, 0.206, 0.314. Blend: $z = 0.480(0.2, 3) + 0.206(1, 1) + 0.314(0.4, 2.2) = (0.43,\ 2.34)$; residual: $(1.43,\ 3.34)$.

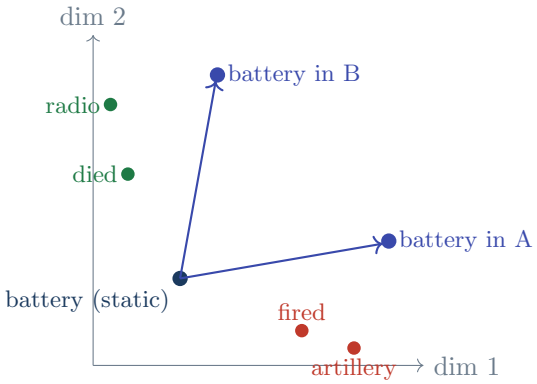


Figure 1: one attention step moves the same static vector in two different directions.

The query and the arithmetic are identical; only the neighbours differ, so the result differs. Measured by cosine similarity, the static vector was equally close to artillery and radio (0.753 each). After one attention step, battery-in-A has cosine 0.946 with artillery and 0.448 with radio; battery-in-B has 0.454 and 0.943. The ambiguity is resolved.

Key point

A contextual embedding is the static vector plus a weighted blend of the other tokens' values. Nothing more mysterious happens in one attention step.

Pitfall — identity matrices are a teaching shortcut

With $W^Q = W^K = W^V = I$ every token asks exactly what it advertises and hands over exactly itself. Learned matrices let a word's question differ from its advertisement and its content; for example a verb's query can seek its subject while its key advertises its tense. Lecture 9 works an example with learned matrices.

8 Multi-Head Attention

One attention computation can capture one kind of relationship at a time. **Multi-head attention** runs h attention computations (heads) in parallel, each with its own W_i^Q, W_i^K, W_i^V, so each asks a different question of the same sentence. Their outputs are concatenated and mixed by one further matrix W^O:

$$\text{MultiHead}(X) = \left[\text{head}_1; \ldots; \text{head}_h\right] W^O.$$

Intuition

a surveillance sensor suite Radar, infrared and a camera watch the same scene. Each picks up what the others miss — range, heat, shape — and the fused picture beats any single sensor. Heads do the same for relationships between words.

The head size is chosen so the heads together match the embedding width: BERT-base $12 \times 64 = 768$; GPT-3 $96 \times 128 = 12{,}288$. So multi-head attention costs about the same as one full-width head. Nobody assigns roles to heads; specialisations emerge in training. Studies of trained models

have found heads that link modifiers to nouns ("enemy"↔"armour"), verbs to their subjects and objects ("spotted" with "drone" and "armour"), and words to locations ("near", "ford"). Real heads are messier: many overlap and some do little. Attention heat-maps are clues to what a model attends to, not proof of its reasoning.

9 Completing the Block

9.1 Residual connections

The attention output is *added* to the vector that went in, $x \leftarrow x + \text{Attention}(x)$, as we did for battery: $(1, 1) + z$. Like an analyst writing in the margin of a report rather than retyping it, the original survives and annotations accumulate. During training the gradient flows straight through the "+" (its derivative is 1), so stacks of dozens of blocks still learn — the same reason residual connections made very deep CNNs trainable in Lecture 7.

9.2 Layer normalisation

Each token's vector is shifted to mean 0 and scaled to standard deviation 1, then multiplied by a learned scale γ and shifted by a learned β:

$$\text{LN}(x) = \gamma \, \frac{x - \mu}{\sigma} + \beta.$$

Worked example — normalising $(2, 4, 6, 8)$

Mean $\mu = 5$; variance $\frac{9+1+1+9}{4} = 5$; $\sigma = \sqrt{5} = 2.236$. Normalised: $(-3, -1, 1, 3)/2.236 = (-1.342, -0.447, 0.447, 1.342)$, with mean 0 and standard deviation 1.

Like calibrating instruments before fusing their readings, normalisation keeps numbers in a stable range layer after layer, making training faster and more reliable.

9.3 The feed-forward network

After attention each token passes, separately, through the same small two-layer network:

$$\text{FFN}(x) = W_2 \, f(W_1 x + b_1) + b_2,$$

with a non-linear f (ReLU or GELU) and a hidden layer about four times wider than d (BERT-base 768 → 3,072 → 768; GPT-3 12,288 → 49,152 → 12,288). Attention *gathers* information from other tokens and is essentially a weighted average; the FFN *processes* what each token gathered, with the non-linearity that Lecture 6 showed is needed for anything beyond linear patterns. In the briefing-room analogy, attention is the staff briefing where everyone hears from everyone; the FFN is each officer back at their desk working through their own notes, by the same procedure and in parallel.

9.4 One block, then stack it

An encoder block is: multi-head attention → add & normalise → feed-forward → add & normalise. Vectors come out the same size as they went in, so blocks stack: the original transformer used 6 per side, BERT-base 12, BERT-large 24, GPT-3 96. Early blocks tend to resolve local word meanings (battery = gun unit); later blocks build sentence- and paragraph-level understanding. Every token goes through every head and the FFN at the same time, which is one reason transformers train fast on GPUs.

Worked example — where BERT-base's 110 million parameters live

Per block: attention $4 \times 768 \times 768 + 4 \times 768 = 2{,}362{,}368$; feed-forward $2 \times 768 \times 3{,}072 + 3{,}072 + 768 = 4{,}722{,}432$; two layer norms 3,072. Block total 7,087,872; twelve blocks 85,054,464. Embeddings: tokens $30{,}522 \times 768 = 23{,}440{,}896$, positions $512 \times 768 = 393{,}216$, sentence segments 1,536, plus a layer norm 1,536: 23,837,184. With the 590,592-parameter pooling layer the total is 109,482,240 $\approx$ 110 million. Two-thirds of each block's weights are in the feed-forward network.

10 The Decoder

10.1 Writing one token at a time

The decoder begins with a ⟨start⟩ token and at each step predicts the next token, which is appended and fed back: ⟨start⟩ → SIZE → 2 → ACTIVITY → moving → north → $\cdots$ → ⟨end⟩. At every step it uses two inputs: the encoder's contextual vectors for the whole report (computed once) and the tokens produced so far. Its output layer gives a probability for every vocabulary token; the most likely or a sampled one is written.

10.2 Masked self-attention

While writing step 4 the decoder may attend to steps 1–3 but not to the future. In training the whole correct output is supplied at once, and a **mask** sets the score of every future position to $-\infty$ before the softmax, so its weight is $e^{-\infty} = 0$. All positions can then be trained in parallel without any position seeing its own answer.

Worked example — masking in sentence B

If "radio battery died" is processed by a decoder, battery (position 2) may see only radio and itself. Scaled scores 2.263 and 1.414 give weights 0.700 and 0.300 (the score for "died" is masked to $-\infty$). Blend: $0.7(0.2, 3) + 0.3(1, 1) = (0.44,\ 2.40)$. The left context alone is enough here to pull battery towards the power-cell meaning.

10.3 Cross-attention

Each decoder block has a second attention layer in which the **queries** come from the decoder ("I am writing the LOCATION field; which part of the report do I need?") and the **keys and values** come from the encoder's vectors for the input. Because Q and K, V come from different sequences this is **cross-attention**. Each output field looks up the words that justify it: LOCATION draws on "near the bridge", TIME on "0600". Translation works the same way, aligning output words with input words. Encoder-only BERT has no cross-attention, and neither does decoder-only GPT, whose prompt and answer form one sequence.

10.4 The whole journey

Report text → tokens and IDs → embeddings + positions → N encoder blocks (self-attention, FFN) → contextual vectors. Output so far → embeddings + positions → N decoder blocks (masked self-attention, cross-attention, FFN) → probabilities → next token, appended and repeated. Every step uses only lookups, additions, matrix products, softmax and a small non-linear network — repeated at scale.

11 Defence Relevance and Cautions

Defence & wargaming application — where transformers help

Contextual meaning: "battery", "fire", "contact" and "engage" change meaning with context, exactly what attention resolves.
Structured reporting: encoder–decoder models turning field notes into SALUTE, SITREP or 9-line formats for a human to check.
Search by meaning: embeddings let analysts find reports about armour that never use the word "tank".
Triage and summarisation: ranking and condensing large volumes of open-source and intelligence reporting for analyst review.

Pitfall — cautions

Hallucination: fluent output is not verified output; models can invent units, grids and times.
Jargon and abbreviations: grid references and acronyms may be split into many tokens and mishandled.
Prompt injection: text inside an input document can try to steer the model.
Explanation: attention maps suggest, but do not prove, why a model answered as it did.
Keep a trained human in the loop for any output that feeds a decision.

12 Common Misconceptions

Pitfall — five errors to avoid

"The model looks facts up." It predicts probable tokens; there is no database unless one is explicitly attached.
"Each embedding dimension means something nameable." Meaning is spread across all dimensions.
"Attention weights say what the model reasoned." They show where information was gathered, not why an output was chosen.
"Transformers read left to right like RNNs." The encoder processes all tokens in parallel; order comes only from position vectors.
"Multi-head attention multiplies the cost by h." Each head works in d/h dimensions, so the total cost is about that of one full-width head.

13 Summary of Equations

Next-token probability	$P(i) = e^{s_i} / \sum_j e^{s_j}$
Cosine similarity	$\cos(\mathbf{a}, \mathbf{b}) = \mathbf{a} \cdot \mathbf{b} / (\|\mathbf{a}\| \|\mathbf{b}\|)$
Input vector	$x_t = \text{embedding}_t + \text{position}_t$
Projections	$q = xW^Q$, $k = xW^K$, $v = xW^V$
Attention	$\text{softmax}(QK^\mathsf{T}/\sqrt{d_k})\, V$
Multi-head	$[\text{head}_1; \ldots; \text{head}_h]W^O$
Residual and norm	$\text{LN}(x + \text{Sublayer}(x))$, $\text{LN}(x) = \gamma(x - \mu)/\sigma + \beta$
Feed-forward	$W_2\, f(W_1 x + b_1) + b_2$
Training loss	$L = -\ln P(\text{correct next token})$

14 Check Your Understanding

Check your understanding — 1

Why can token ID numbers not be used directly as meanings?

Answer. IDs follow the order of the vocabulary (often spelling or frequency), so numerically close IDs need not have close meanings, and arithmetic on IDs is meaningless. Embeddings place tokens so that distance and direction reflect usage.

Check your understanding — 2

In sentence B, which word gave battery the most attention weight, and how much?

Answer. "Radio", with weight 0.480 (score 3.2, scaled 2.263, exponential 9.609 out of a total 20.009).

Check your understanding — 3

In the radio-net analogy, what plays the role of the key, and what is it in the mathematics?

Answer. The call-sign each station transmits; mathematically the key vector $k = xW^K$ of each token, which is matched against the listener's query by a dot product.

Check your understanding — 4

Why does the decoder need a mask during training?

Answer. In training the whole target output is fed in at once so that all positions are trained in parallel. Without the mask each position could attend to the token it is supposed to predict and would learn to copy it. The mask sets future scores to $-\infty$, giving them zero weight.

Check your understanding — 5

In cross-attention, where do the queries come from, and where do the keys and values come from?

Answer. Queries from the decoder (what the output being written needs); keys and values from the encoder's contextual vectors for the input.

Check your understanding — 6

Scores $(2, 1, 0)$ are computed with $d_k = 4$. What are the attention weights?

Answer. Divide by $\sqrt{4} = 2$: $(1, 0.5, 0)$. Exponentials 2.718, 1.649, 1, sum 5.367. Weights (0.506, 0.307, 0.186).